UNIT 3, AGENT DEVELOPMENT WITH PYTHON AND FRAMEWORKS

LECTURE 5

Multi-Agent Systems, the Graph Model

The finale of the unit. One agent becomes several that pass work between them, and the old conversation-first model gives way to something you can actually see, test, and trust: a graph.

CSE476 | Rohit Bharti

SECTION 00

Through the door

In Lecture 3 you saw it. Today you walk through.

RECALL

The door from Lecture 3

```
manager = client.as_agent(
    instructions="Delegate to your specialists.",
    tools=[triage_agent.as_tool(), ...])
#           ^ an agent, used as a tool by another agent
```

ONE LINE CHANGED EVERYTHING

A tool can be a function, or it can be another whole agent. That single idea, agent.as_tool(), turns one agent into a team. Today we build that team two ways, the simple way that works for small jobs, and the powerful way that real systems are built on.

Today

- The simple multi-agent shape: a manager that delegates to specialists.
- Where that shape hits its ceiling, and why real systems need more.
- The graph model: nodes that do work, edges that route between them.
- The shift that defines the whole framework merger: conversation, to graph.
- A real Agent Framework graph, built and run, routing tickets with no guesswork.

HOLD ON TO THIS

This closes Unit 3. By the end you will have built a real multi-agent graph, seen exactly why it beats a free conversation between agents, and be able to explain that difference in an interview, which is where this knowledge pays.

SECTION 01

The simple way

A manager agent delegating to specialists.

A manager over specialists

```
triage  = client.as_agent(name="triage",  instructions=...)
billing = client.as_agent(name="billing", instructions=...)

manager = client.as_agent(
    instructions="Delegate to the right specialist.",
    tools=[triage.as_tool(), billing.as_tool()])
```

THE MANAGER DECIDES WHO HANDLES WHAT

Each specialist is an ordinary agent, good at one thing. The manager is an agent whose tools happen to be other agents. Ask it something, and it delegates to the right specialist and returns their answer. For a small team and a simple flow, this is genuinely all you need, and it is only a few lines.

Where the manager hits its ceiling

WHAT THE MANAGER DOES WELL

Simple delegation

Two or three specialists, a request comes in, route it to one, return the answer. Clean and easy, and the manager holds the whole plan in its head.

WHERE IT STRUGGLES

Real structure

Steps that must run in a fixed order. Branches that depend on a result. Work that fans out to several specialists and rejoins. Now the plan is complex, and it lives only in one model's head, invisible and unenforced.

THE PROBLEM WITH A PLAN IN A MODEL'S HEAD

You cannot see it, cannot test it, and cannot guarantee it runs the same way twice. When the flow has real structure, you want that structure written down and enforced by the system, not remembered by a model that might improvise. That is exactly what the graph gives you.

SECTION 02

The graph model

The shift that defines the whole framework merger.

CONVERSATION, TO GRAPH

The shift, stated plainly

AUTOGEN, THE OLD WAY

Agents hold a conversation

The original multi-agent idea: put agents in a room, let them talk to each other, and hope the conversation converges on an answer. Powerful when it works, and impossible to see inside, test, or reproduce when it does not.

AGENT FRAMEWORK, THE NEW WAY

An explicit graph

Draw the flow as a graph. Nodes do work, edges say what happens next. The control flow is written down, visible, and enforced by the framework. You can read it, test it, and trust it to run the same way every time.

THIS IS THE MOST IMPORTANT SLIDE FOR AN INTERVIEW

When someone asks why Semantic Kernel and AutoGen merged, this is the answer that shows you understand it. AutoGen's free conversation was hard to make reliable. The graph makes multi-agent control flow inspectable and testable, which is what production needs. Say that, and you sound like someone who gets it.

A graph is nodes and edges

```
@executor(id="classify")
async def classify(ticket, ctx):
    queue = ...                  # do the work
    await ctx.send_message(queue) # send it onward

workflow = (WorkflowBuilder(start_executor=classify)
    .add_edge(classify, enrich)   # classify THEN enrich
    .add_edge(enrich, assign)     # enrich THEN assign
    .build())
```

READ THE EDGES AND YOU SEE THE WHOLE FLOW

A node does one step of work and sends a message onward. An edge says which node runs next. The entire control flow is those add_edge lines, right there in front of you, not hidden in a model. This is your Unit 2 pipeline, promoted to a real graph the framework runs.

The graph, drawn

THIS PICTURE IS THE CODE

The three boxes are your three executor functions. The two arrows are your two add_edge lines. When you can draw your system as a diagram and the diagram IS the code, you have something you can reason about, hand to a colleague, and test node by node. That is what the graph buys you over a conversation nobody can draw.

And it runs, with no model at all

```
>>> await run_triage_graph("I was charged twice, refund")
"Assigned to billing-team, 24h SLA, priority normal."

>>> await run_triage_graph("someone hacked my account")
"Assigned to abuse-team, 1h SLA, priority urgent."
```

DETERMINISTIC, TESTABLE, NO TOKENS SPENT

Because these nodes are plain functions, the whole graph runs offline and gives the same answer every time. You can unit test the entire flow with no model and no cost, then swap any node for an agent when you need real judgement at that step. Structure you can test, intelligence where you need it. That is the shape of a reliable multi-agent system.

SECTION 03

Closing the unit

What you can now do, and where it goes next.

The graph, mapped to what you built

`Executor / node`	→	a step of work, like a Unit 2 pipeline step
`Edge`	→	the order you wrote as pipeline sequence
`WorkflowBuilder`	→	your Pipeline builder, now a branchable graph
`ctx.send_message`	→	passing state on, your pipeline state dict
`ctx.yield_output`	→	the pipeline's final return value
`fan-out and fan-in`	→	one input to many workers and back: new

Hold on to this

A conversation you hope converges, or a graph you can see and test. Choose the graph.

An agent can be a tool	A manager delegating to specialists. Simple, and often enough.
That plan lives in a model	Invisible and unenforced. Fine until the flow has real structure.
A graph makes flow explicit	Nodes do work, edges route. You can read, test, and trust it.
Conversation to graph	The heart of the merger, and the answer an interviewer wants.

Before next session

DO	**Build the graph**	Run the triage graph. Then add a fourth node, for example a notify step after assign. Notice you add one edge and the flow updates, with the whole path still readable in the builder.
DO	**Make a node an agent**	Replace the classify function node with a real agent that classifies. The rest of the graph does not change. See how structure and intelligence combine cleanly.
THINK	**Draw before you build**	For your capstone, draw the multi-agent flow as a graph on paper first. If you cannot draw it, you cannot build it reliably. The drawing is the design.

ON THE THIRD ONE

If your capstone has more than one agent, draw the graph before you write a line. A flow you can draw is a flow you can build, test, and explain. A flow you cannot draw is a conversation you are hoping will work, and hope is not an engineering strategy.

End of Unit 3

Unit 4, Multi-Agent Systems and Collaboration

You can now build on real frameworks: a shippable agent, memory, retrieval, and a multi-agent graph. Unit 4 goes deep on the multi-agent world you just opened: patterns of collaboration, agents that specialise and negotiate, and orchestration at a scale where the graph model earns its keep. You built the foundation by hand and the framework on top of it. Now you compose.

COME WITH

Your extended graph, one node turned into an agent, and a hand-drawn graph of your capstone flow.